

Privileged Process Credit Assignment for Codeforces RLVR

Sergei Kopylov

Research project report

June 2026

Project supervisor: Ivan Vovk

Adapted from the original research report for the Higher School of Economics.

Contents

1	Introduction	1
2	Problem Statement	4
3	Methods	5
3.1	Sparse Privileged OPD Teacher	5
3.2	Privileged Value Function	5
3.2.1	Idea	5
3.2.2	Implementation	5
3.2.3	Results	6
3.2.4	Credit Assignment Results	9
4	Literature Review	12
5	Plans	13
6	Conclusion	14
	Appendix: Sparse Privileged OPD Teacher	16

1 Introduction

Codeforces solutions can be checked automatically. A model generates a reasoning trace and a final C++ solution, which is then compiled and run against tests. If it passes verification, the rollout receives reward = 1. If the solution fails to compile, fails a test, or produces an incorrect answer, the rollout receives reward = 0. This makes reinforcement learning with verifiable rewards (RLVR) particularly attractive: the reward requires no manual annotation and is directly connected to solution correctness.

This setting has an important limitation. The model generates more than a single action. It produces a long trajectory comprising reasoning, algorithm selection, case analysis, code, and corrections. The verifier reports only the final outcome; it does not identify which parts of the process contributed to that outcome.

Standard group relative policy optimization (GRPO) uses the final signal as an advantage for the entire trajectory. When reward = 1, the update raises the probability of every token in the rollout: important solution steps, unnecessary reasoning, and incidental stylistic phrases. When reward = 0, it lowers the probability of the entire trajectory, including useful partial ideas that may have preceded a later error.

For reasoning LLMs, this creates a difficult credit-assignment problem. Reasoning trajectories in our Codeforces experiments can be tens of thousands of tokens long, sometimes exceeding 30,000. Judging each fragment separately is impractical: there are too many local decisions, and the assessment can become nearly as difficult as solving the problem. Even a strong frontier model may struggle to distinguish a genuinely correct reasoning step from a plausible but mistaken one, especially when the value of a fragment depends on later context.

The project’s goal is therefore to find a practical process-credit signal for long reasoning trajectories, rather than merely another whole-trajectory classifier. Such a signal should help locate the contribution of individual fragments to the verifiable outcome: where the model made an important algorithmic decision, where it followed a wrong plan, and where the text was largely neutral. The signal should be sparse [4]. The aim is to identify a small number of useful spans, not to annotate every token by hand.

Early Codeforces experiments provide the practical motivation. The baseline was a Qwen3-8B RLVR run with a long generation budget: the model sampled reasoning and a final solution, the testing system returned a binary reward, and GRPO updated the model from these verifiable outcomes. Yet the baseline analysis indicated that this signal alone did not lead to stable improvement on the Codeforces tasks studied here.

If binary reward by itself were sufficient, we would expect validation accuracy to grow steadily. We would also expect some initially hard tasks to become solvable: the model would find the right algorithm more often, correct typical errors, and obtain positive rollouts where previous attempts all received reward = 0. Both effects were weak in practice. Validation quality showed little sustained growth, and the additional diagnostics showed little movement of hard tasks into the easy group.

Three initial experiments were run. Two served as sanity checks of the training and evaluation pipeline. The *192 Train / 192 Val x4* variant shows metric behavior when training and validation reuse the same tasks. The *8k Max Output* variant tests the effect of a shorter response limit. These are diagnostic controls, not evidence of improved process credit.

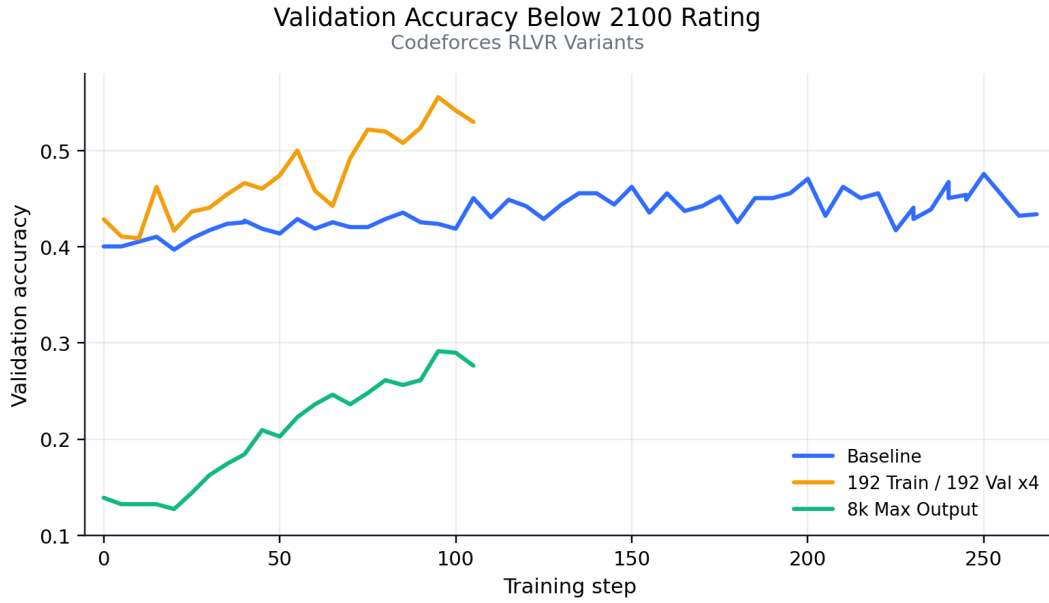


Figure 1: Validation accuracy on Codeforces tasks. The baseline curve remains nearly flat: there are local fluctuations, but no sustained improvement in validation quality. This is an initial sign that binary reward alone is insufficient for reliable learning on these Codeforces tasks.

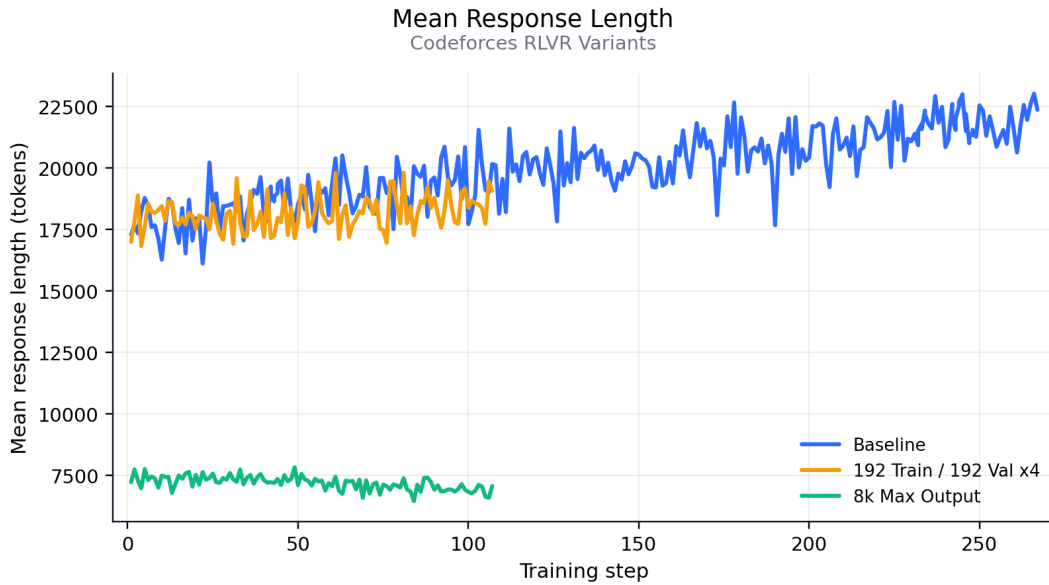


Figure 2: Mean response length during RLVR training. The baseline gradually generates longer answers, whereas *8k Max Output* has a shorter budget. This helps distinguish a possible reward gain from a mere increase in reasoning length.

Together, these plots show that the baseline spends more tokens without a corresponding sustained improvement in validation accuracy.

The third plot breaks down accuracy by task group. Each task is evaluated using 16 rollouts. A rollout is positive if its generated solution passes verification and receives reward = 1; otherwise it receives reward = 0. In this diagnostic, a hard task has zero successful rollouts among the 16 attempts. An easy task has at least one successful rollout, so the model can already find a correct solution occasionally.

The *learnable* and *unlearnable* groups describe what happens to initially hard tasks later in the observed run. Some acquire successful rollouts and improve; others remain near zero accuracy. These labels describe the observed behavior, not an absolute limit on whether a task could ever be learned.

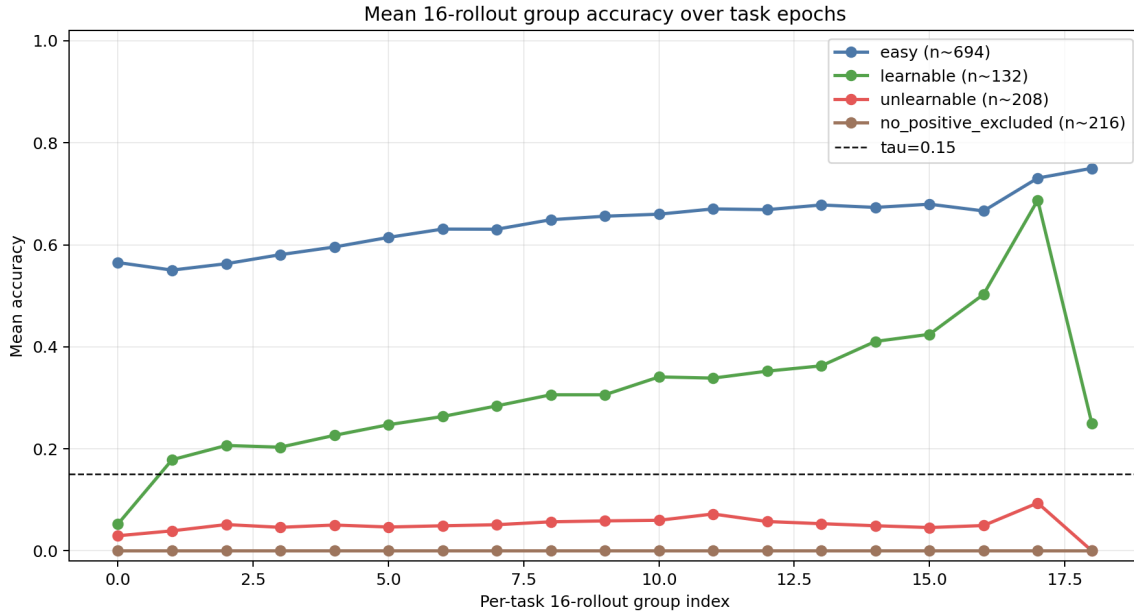


Figure 3: Mean accuracy by task group, estimated from 16 rollouts per task. Easy tasks remain the most solvable; learnable tasks improve partly, whereas unlearnable and no-positive-excluded groups remain near zero. The profile shows little sustained transition from hard to easy during baseline training.

The dashed line marks $\tau = 0.15$, an accuracy level at which a task has a noticeable fraction of successful rollouts. If the baseline were mastering many new task groups, more hard tasks would move toward the learnable or easy region. This transition is limited in the observed run.

The baseline thus supplies a useful global signal but weak local credit assignment. The verifier says whether the final solution passed, not where the model chose the right algorithm, made the critical mistake, or produced reasoning that was useful or unnecessary. GRPO applies one trajectory-level advantage to all tokens. In successful rollouts, this can reinforce detours and stylistic patterns along with important decisions. In failed rollouts, it can suppress correct observations or a useful invariant that was followed by a later error.

One binary outcome therefore mixes helpful and harmful actions within the same update. Similar limitations are discussed in *The Unlearnability Phenomenon in RLVR for Language Models* [1], which examines initially hard examples that remain difficult under RLVR and points to weak gradient agreement and poorly generalizing reasoning patterns.

The working hypothesis of this project is that final verifier reward should be supplemented with a sparse process-credit mechanism. The final reward provides the global direction, while a teacher helps locate the important local decisions. The student continues to see only the public problem statement. During training and analysis, the teacher may see privileged information such as an editorial, correct solution, or reference code, and evaluate the trajectory that the student actually produced.

2 Problem Statement

Let x denote the public problem statement, and let z denote privileged solution information. The latter could be an editorial, verified C++ code, a reference implementation, or other knowledge about the particular task.

The student generates a trajectory using only the public statement:

$$y = (y_1, \dots, y_T), \quad y \sim \pi_{\text{student}}(\cdot \mid x).$$

The final verifier returns a label $l \in \{0, 1\}$, where $l = 1$ means accepted and $l = 0$ means rejected.

The roles of the models must remain separate. The student $\pi_{\text{student}}(\cdot \mid x)$ does not see z . A teacher may inspect the student’s prefix and the privileged information:

$$\pi_{\phi}(\cdot \mid x, z, y_{<t}).$$

The goal is a practical signal for process credit assignment. Here, the teacher is an auxiliary mechanism rather than one fixed architecture; the following sections consider several ways to implement it. Its common purpose is to extract evidence about local reasoning steps from the task statement, privileged information, and the already generated student trajectory: which fragments support the eventual solution, which are associated with an error, and which are largely neutral. This evidence can help allocate the final verifier reward more locally instead of assigning it uniformly to the whole trajectory.

3 Methods

3.1 Sparse Privileged OPD Teacher

A large part of the project was spent on the sparse privileged OPD teacher. The goal was to use privileged information not to generate a solution, but to score the student’s own trajectory locally and provide process-credit evidence, in the spirit of on-policy distillation and privileged on-policy self-distillation [5, 6, 3].

This direction did not become the main successful result. The teacher learned a weak ranking signal between accepted and rejected trajectories, but held-out validation remained moderate, and I could not reliably show that sparse movement appears at the actual reasoning-critical spans.

For this reason, I include the detailed technical write-up at the end of this PDF document. It is kept in its original English form and should be read as an appendix-style work log for the sparse-OPD direction.

3.2 Privileged Value Function

3.2.1 Idea

The second direction was a privileged value function. Instead of comparing token distributions, this method trains a scalar model

$$V_{\theta}(x, y_{\leq t}) \approx \Pr(\text{reward} = 1 \mid x, y_{\leq t})$$

at several points of the generated reasoning trajectory. Here x is the Codeforces problem and $y_{\leq t}$ is the prefix of the student’s answer.

The motivation is close to implicit process reward modeling [9]. If the value changes sharply after a particular reasoning segment, this segment may be useful evidence for credit assignment. A positive change does not prove that the local step is mathematically correct, and a negative change does not prove that the local step is wrong. It only says that, under the trained teacher, the prefix became more or less likely to end with `reward=1`.

This makes the value function weaker than a real process verifier, but also much easier to train. It can be used as an audit tool for long trajectories: instead of asking a model to judge every span independently, we observe where the predicted probability of final success moves.

3.2.2 Implementation

The value teacher was trained on Codeforces trajectories generated by the student model. For each trajectory, the final verifier label was known: `reward=1` for accepted solutions and `reward=0` for rejected solutions. The same label was attached to sampled prefixes of the trajectory, and the model was optimized with binary cross-entropy on these sampled value points.

In the main run, the sampled points were newline-based. The model saw the student’s partial answer at each newline point and predicted a scalar success probability. The terminal value was also logged, because it is the easiest place to check whether the model distinguishes accepted and rejected trajectories.

I tested two prompt variants. In the public variant, the teacher saw only the public problem statement and the student’s answer. In the privileged variant, the teacher additionally saw editorial information during training and validation. The privileged information was not used to generate the

student’s solution; it was used only to train the scalar evaluator of an already generated trajectory. I refer to the public variant as the public-prompt ablation: it keeps the same data, model, objective, and validation split, but removes the editorial part of the prompt.

The main diagnostics were:

- terminal pairwise AUC and terminal threshold accuracy;
- mean terminal probability for accepted and rejected trajectories;
- prefix BCE validation loss on sampled trajectory prefixes.

3.2.3 Results

The first result is that the privileged value function learned a non-trivial terminal ranking signal on random-trajectory validation. The pairwise AUC grew quickly, while threshold accuracy was less smooth. This is expected: ranking can improve before the absolute probability scale becomes well calibrated.

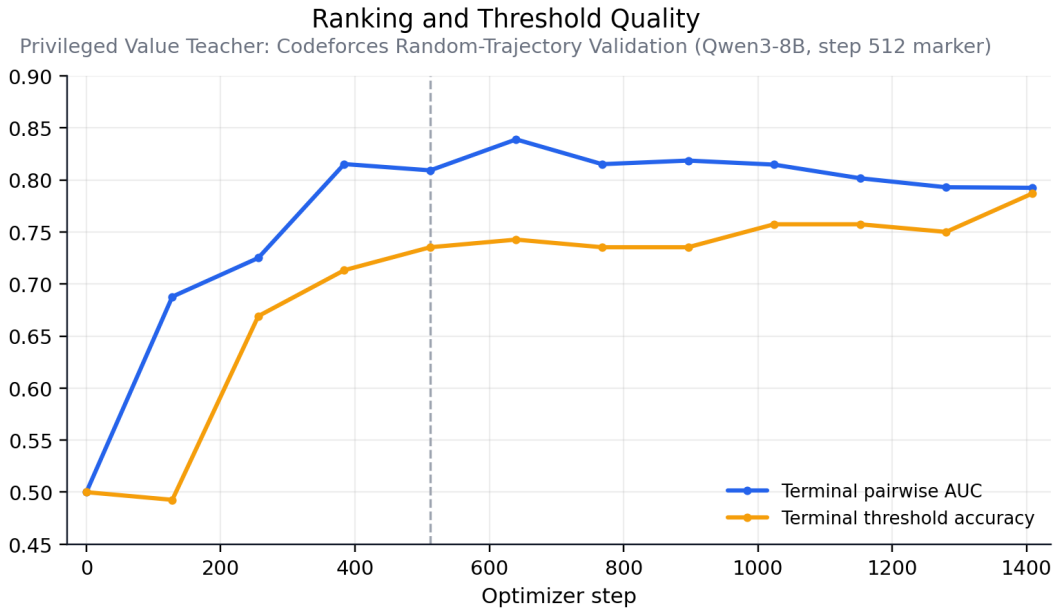


Figure 4: Random-trajectory validation for the privileged value teacher. Terminal ranking improves early, while threshold accuracy is a stricter calibration-dependent metric.

The separation is also visible directly in terminal probabilities. Accepted trajectories receive higher terminal values, while rejected trajectories move toward lower values. This is a necessary sanity check before using the value trace as a process-credit signal.

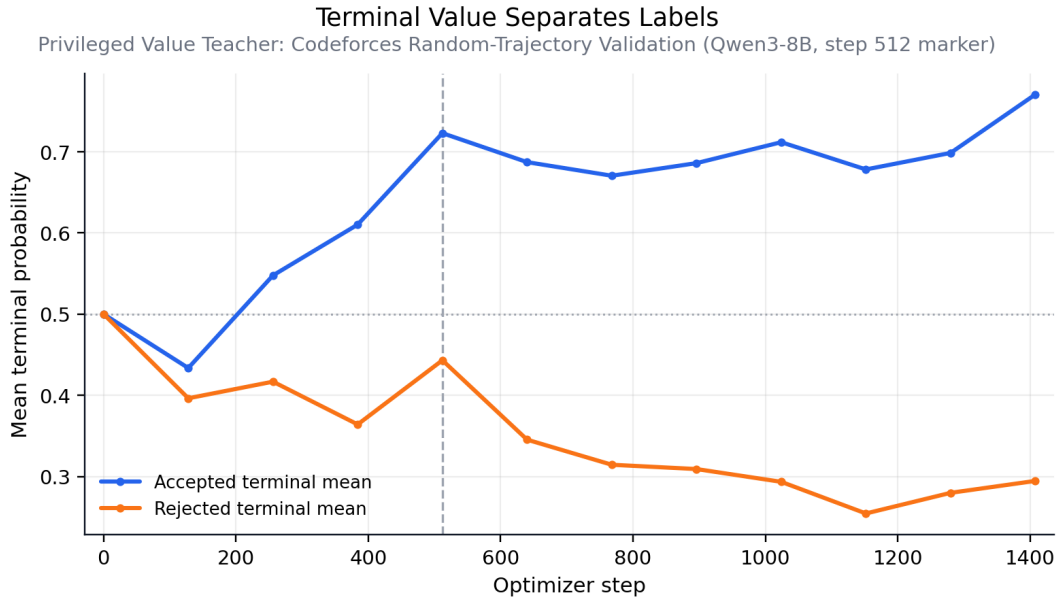


Figure 5: Mean terminal value for accepted and rejected trajectories. The gap shows that the teacher is not merely producing a constant score.

At step 512, the terminal probability histogram has visible separation but still has overlap between accepted and rejected rows. This is exactly the regime where the value function is useful for analysis, but not yet a reliable standalone verifier.

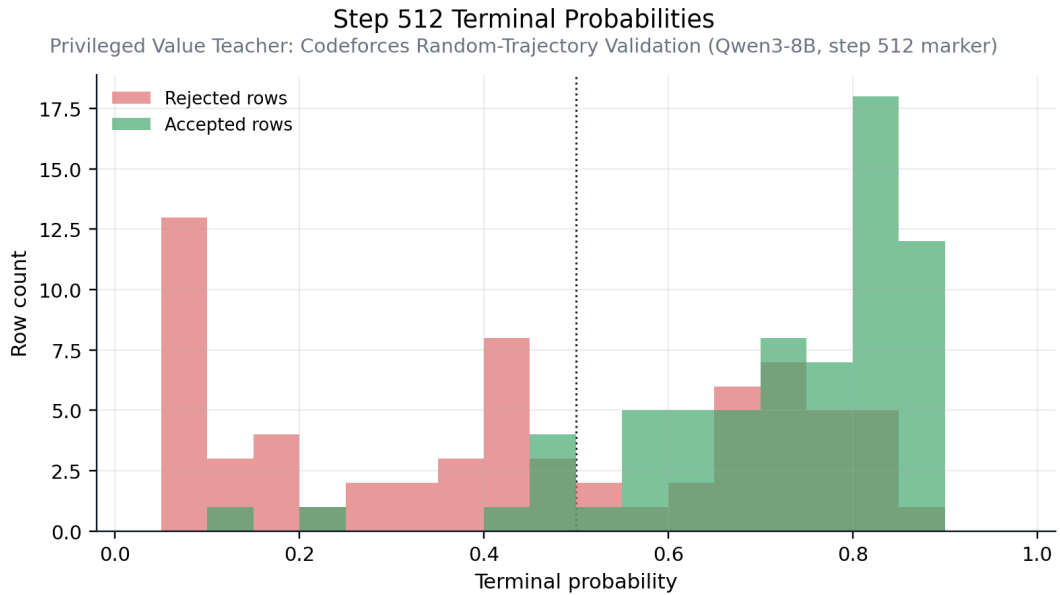


Figure 6: Terminal probabilities at step 512. The accepted and rejected distributions separate, but overlap remains.

Finally, I compared the privileged variant with this public-prompt ablation on the same random-trajectory validation split. This is the cleanest empirical result for the value-function direction. The privileged prompt reaches a much higher terminal threshold accuracy and keeps a clearly lower prefix BCE validation loss over the shared training range. In other words, access to editorial information makes the scalar value teacher substantially better at predicting whether a partial

trajectory will end with `reward=1`.

This result is related to implicit PRM work, where process signals are derived from outcome-level supervision [9], and to privileged self-distillation work, where privileged context is used as a dense teacher signal [6]. I did not find prior LLM work that trains a scalar value function with privileged editorial or reference-solution information for partial reasoning trajectories. Therefore, the public-prompt-versus-privileged comparison here is not a reproduction of a standard ablation, but a clean project-specific result: privileged information substantially improves the value teacher on both terminal threshold accuracy and prefix BCE validation loss. This still does not by itself prove that value deltas are faithful local correctness labels.

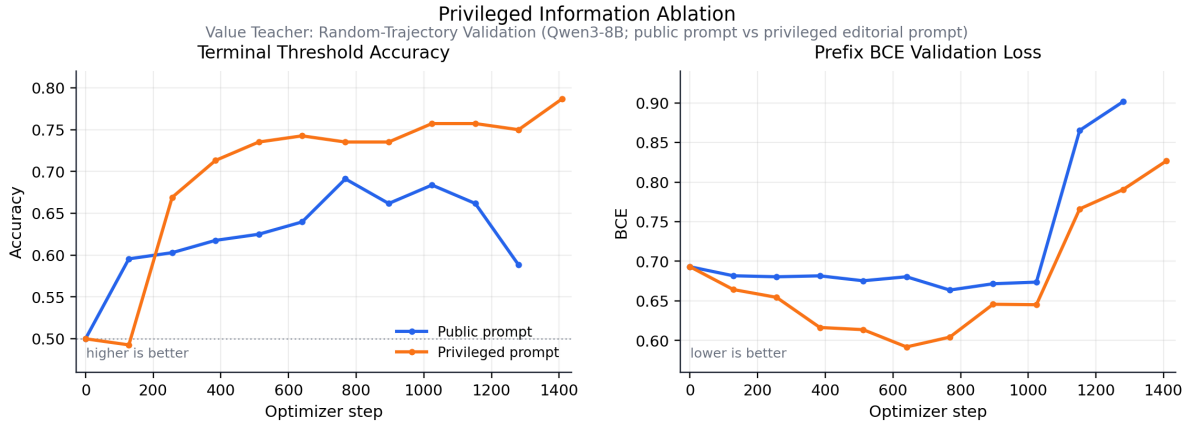


Figure 7: Public-prompt vs privileged-prompt value teacher on random-trajectory validation. The comparison uses terminal threshold accuracy and prefix BCE, the two most direct scalar diagnostics for this value-function setup.

3.2.4 Credit Assignment Results

The scalar metrics above answer whether the value teacher predicts final success better with privileged information. They do not answer the more important process question: do local value changes highlight useful reasoning steps? I manually reviewed five step-512 heatmap fragments. Green means that the predicted probability of final acceptance increased after the span; red means that it decreased. I read this as soft process-credit evidence, not as a ground-truth correctness label.

The clearest positive example is 1175C. The value teacher stays negative while the trajectory is stuck in a brittle case split, and becomes positive when the reasoning switches to the radius/window abstraction. This is a useful process-credit hit. The caveat is that the red region also contains useful self-diagnosis, so it should not be read as purely wrong reasoning.

Scenario B: $k+1 > i$. In this case, the $(k+1)$ th distance is the $(k+1-i)$ th smallest distance among the right distances, which is $a_{i+(k+1-i)} - x = a_{k+1} - x$. To minimize this, we choose x as large as possible in the interval, which is a_{i+1} .

But how to determine which scenario applies?

Let me take an example. For the first sample input, $n=3$, $k=2$, $i=2$ (since x is in $[a_2, a_3]$). $i=2$, $k+1=3$. Since $3 > 2$, we are in scenario B. So the $(k+1)$ th distance is $a_{k+1} - x = a_3 - x$. To minimize this, x should be as large as possible, which is a_3 . But when x is a_3 , the distances are $4, 3, 0$. The sorted distances are $0, 3, 4$. The $(k+1)$ th element is 4. But earlier, we saw that choosing $x=3$ gives 2 as the third element. So why is this scenario not giving the correct result?

Because this scenario assumes that all left distances are less than the minimum right distance, which is not the case when x is a_3 . For $x = a_3$, the left distances are $a_3 - a_1 = 4$, $a_3 - a_2 = 3$. The minimum right distance is 0 (since x is at a_3). So the left distances are all greater than the right distances. So the sorted list of distances includes 0 (the right distance), 3, 4. Hence, the $(k+1)$ th element is 4. But there exists a x in the interval $[a_2, a_3]$ where the $(k+1)$ th element is smaller. So this suggests that scenario B is not the correct way to model it.

This indicates that my previous approach is missing something. Perhaps the $(k+1)$ th distance is not always determined by the scenario A or B, but rather depends on a balance between the left and right distances.

Another idea: the $(k+1)$ th distance is the minimal value d such that there are at least $k+1$ points within distance d of x . So, to find x that minimizes this d .

This is similar to the problem of finding the smallest d such that there are at least $k+1$ points within d of x . The smallest d is the minimal value such that there exists an x where at least $k+1$ points are within d of x . And we need to choose x to minimize d .

But how to find this x ?

This sounds like the problem of finding the smallest d where there exists an x such that at least $k+1$ points are within d of x . But how?

This is similar to finding a point x such that at least $k+1$ points are within distance d , and d is as small as possible.

The minimal d would be determined by the $(k+1)$ th point in the array. For example, if we choose x to be the median of the $k+1$ points. But how?

Let me think: if we select x such that it is the median of the $(k+1)$ th points, then the maximum distance to these points is minimized.

For example, for the first sample input, $k=2$, so $k+1=3$. We need to select x such that there are 3 points within d of x . The minimal d is achieved when x is in the middle of the three points. Which is the case where x is the midpoint of a_1 and a_3 , as in the sample.

So, for general k , the optimal x is chosen such that it is the median of the $(k+1)$ th points in the array.

Wait, but how to determine which points to select?

Let me think: to have at least $k+1$ points within d of x , the minimal d is achieved by choosing x as the median of the $(k+1)$ th points. For example, if we take the (k) th, $(k+1)$ th, and $(k+2)$ th points, or something like that.

Wait, this is getting abstract. Let me try to think for the first scenario.

Figure 8: 1175C credit-assignment case. The value teacher assigns positive movement when the reasoning changes from a brittle scenario split to the radius/window formulation.

The 1473D case shows that value deltas can reward local progress inside a rejected trajectory. The highlighted fragment is a useful derivation, and the teacher marks it green. The final answer still fails later, so this is evidence for local process credit, not evidence that the whole solution is correct.

Let me think back to the original problem. Perhaps there's a way to represent the entire path as a single sequence of prefix sums and then find the number of distinct values.

But how to do this for any query?

Let me think of it as a range query. For a query, the path is the concatenation of two parts: the first part and the second part.

The prefix sums for the entire path are:

$$S_0 = 0$$

$$S_1 = S[1]$$

...

$$S_{l-1} = S[l-1]$$

$$S_l = S[l-1] + (S[r+1] - S[r]) = S[l-1] - S[r] + S[r+1]$$

$$S_{l+1} = S[l-1] - S[r] + S[r+2]$$

...

$$S_n = S[l-1] - S[r] + S[n]$$

So, the total number of distinct values is the number of distinct values in the set $\{ S_0, S_1, \dots, S_{l-1}, S[l-1] - S[r] + S[r+1], \dots, S[l-1] - S[r] + S[n] \}$.

But this is the same as the set $\{ S_0, S_1, \dots, S_{l-1} \} \cup \{ S[l-1] - S[r] + S[k] \mid k \text{ in } r+1 \dots n \}$.

Which is what we had earlier.

Now, note that $S[k]$ for $k \text{ in } r+1 \dots n$ is the same as the prefix sums from $r+1$ to n . Let me call this set T .

Figure 9: 1473D credit-assignment case. Positive value deltas mark a correct shifted-prefix derivation, even though the final trajectory is rejected.

The first 1660B example is the most direct match between value movement and a core idea. The green span contains the second-maximum rule, which is the key condition for the problem. This is the desired behavior: the value teacher highlights the compact algorithmic insight rather than only the final answer.

This would explain why the third sample is NO.

So the condition is:

1. $m \leq (\text{total} + 1)/2$.
2. The number of types with count $\geq (m - 1)$ is at least 2.

Let me check this for the third sample.

$m=6$, $m-1=5$. Number of types with count ≥ 5 is 1 (type2). So condition 2 is not met. Hence, the answer is NO.

Let me check for the first sample:

Figure 10: 1660B credit-assignment case. The green span captures the second-maximum rule, which is the key algorithmic condition for the problem.

The next 1660B fragment shows an important weakness. The rule is already present near the beginning, but the strongest green spans appear only after sample checks. In this case, value credit is delayed: the teacher rewards evidence that the idea works more strongly than the first abstract statement of the idea.

Let's think about the following approach: to check whether the maximum frequency is greater than $(\text{total} + 1)/2$. If it is, then answer is NO. Otherwise, check whether the maximum frequency is greater than the second maximum frequency + 1. If yes, then answer is NO. Otherwise, answer is YES.

Wait, let's see.

For the third example, the maximum frequency is 6, the second maximum is 4. So $6 > 4 + 1$ (since $4+1=5$, $6>5$). So according to this condition, answer is NO. Which matches the example.

Let's test this with the first example: the maximum is 3, second maximum is 2. 3 is not greater than $2 + 1$ ($3>3$ is false). So condition holds, answer YES.

Test case where max is $\text{sum_rest} + 1$ and there's only one type: for example, $a=[3,1]$. sum is 4. $(4+1)/2 = 2.5$. max is $3 > 2.5$ → condition fails. So answer NO.

Another example where max is $\text{sum_rest} + 1$ but with two types:

$a=[3,3]$. sum is 6. $(6+1)/2=3.5$. max is $3 \leq 3.5$. So condition holds. The second maximum is 3. 3 is not $> 3 + 1$. So answer is YES.

Figure 11: 1660B delayed-credit case. The rule is already present at the top, but the strongest positive value movement appears on subsequent sample checks.

The last 1660B fragment shows the opposite limitation. The main route is wrong, so the negative movement is broadly reasonable. However, some sentences inside the red region notice a real constraint of the problem. This is a coarse-credit failure: a wrong route can still contain useful intermediate observations.

but how to ensure that such a sequence exists?

The problem is similar to the problem of rearranging characters to form a string with no two consecutive identical characters. In that problem, the condition is that the maximum frequency is not more than $(\text{total} + 1)/2$.

So perhaps this problem is similar, but with the additional constraint that in each step, you must choose a type with the maximum count, not just any type.

But in this case, the solution may be similar. So the necessary and sufficient condition is the maximum frequency not exceeding $(\text{total} + 1)/2$. But why in the third example, the answer is NO, even though the condition holds?

So, the problem is that even though the condition holds, there is no way to arrange the sequence such that during the process, when you have to choose from the most frequent type(s), there's always an option to choose a different type.

So, perhaps the answer is YES if and only if the maximum frequency is $\leq (\text{total} + 1)/2$, and there are at least two types with maximum frequency or the maximum frequency is not greater than the sum of the other frequencies.

Wait, but earlier example with $a = [3, 1]$, max is 3, sum_rest is 1. So max is $3 > \text{sum_rest} + 1$ ($1+1=2$). So in this case, the condition holds, but the answer is NO. So the condition $\text{max} \leq (\text{sum} + 1)/2$ is sufficient.

But this suggests that the third example is a counterexample.

But why? Let me recheck.

In the third example:

Figure 12: 1660B noisy-detour case. The negative value movement correctly penalizes the ordinary rearrangement route, but it also covers useful diagnostic sentences.

Overall, the qualitative result is positive but limited. The value teacher can highlight meaningful events: pivots, useful derivations, and validation checks. Importantly, green local steps inside rejected trajectories are not a failure by themselves: this is exactly the kind of local progress signal that process credit should be able to capture. The limitations are different: credit can be delayed, too coarse, or mixed with nearby incorrect reasoning. I would therefore use value deltas as an audit and candidate-selection signal, not as direct ground-truth supervision for RLVR.

For the value-function direction as a whole, the evidence is stronger than for the sparse OPD teacher. The model learned to distinguish accepted and rejected trajectories, benefited strongly from privileged information, and produced several qualitatively meaningful process-credit highlights. The main limitation is conceptual: because the model is trained from final labels on sampled prefixes, it is still an implicit process signal, not ground-truth step supervision.

4 Literature Review

This project is closest to four lines of work.

Process supervision and PRMs. The standard motivation for process reward models comes from process supervision: instead of judging only the final answer, a model can be trained to evaluate intermediate reasoning steps [4]. This is the right conceptual target for Codeforces credit assignment, but direct PRM-style supervision is expensive here. A single Codeforces trajectory can be tens of thousands of tokens long, and reliable local labels would require understanding both the algorithm and the implementation details.

Implicit process rewards. Implicit PRM work shows that process-like signals can be derived from outcome-level supervision without explicit step labels [9]. PRIME takes a related online direction and uses implicit process rewards for RL training from policy rollouts and outcome labels [2]. These papers are important for this report because the value-function direction also uses final verifier labels to obtain a softer prefix-level signal. The difference is that this project additionally tests whether privileged editorial information helps the scalar value teacher.

Privileged and on-policy distillation. On-policy distillation studies teacher feedback on student-generated trajectories [5]. Self-Distilled RLVR and EDGE-OPD are especially relevant because they use privileged context during training while the student remains public at inference time [6, 3]. This is the closest family of ideas to the sparse privileged OPD teacher. It also motivates the project constraint used throughout this report: privileged information should help assign credit, but it should not become a hidden shortcut for generating the student’s final solution.

RLVR limitations and dense process feedback. Recent RLVR work emphasizes that outcome rewards alone can be misleading. The unlearnability result argues that some initially hard prompts do not become learnable under standard RLVR dynamics [1]. Other work shows that outcome rewards do not necessarily make reasoning traces causally important or self-sufficient [7]. Verifiable Process Rewards study the opposite regime, where intermediate actions can be checked by reliable symbolic or algorithmic oracles [8]. These results support the main premise of this project: for long Codeforces trajectories, final verifier reward is useful but too sparse, so additional process-credit evidence is needed.

Overall, the literature provides the ingredients: process supervision, implicit process rewards, privileged distillation, and RLVR failure analyses. The project-specific contribution is to test these ideas in Codeforces RLVR and, in particular, to compare public-prompt and editorial-privileged value teachers on the same random-trajectory validation split.

5 Plans

The next step is to connect the value-teacher direction with the student’s internal representations. The current value teacher reads only generated tokens and predicts a scalar success probability. It does not see the hidden states that produced those tokens, although these states may contain useful algorithmic information before it is written explicitly.

Adding student hidden states to the teacher. One extension is to give the teacher both the textual prefix and the student’s hidden state at the same position. The hidden state can be used as an additional feature for predicting `reward=1`, value deltas, or local process-credit labels. The student still remains unprivileged at inference time; privileged information is used only to train or validate the teacher signal.

The 1660B hidden-state probe. We already tested a small version of this idea on Codeforces 1660B. The target was the first explicit appearance of the top-two idea: the maximum frequency should not exceed the second maximum by more than one. A frozen-backbone linear probe over Qwen3-8B hidden states was trained on 59 trajectories and 1309 probe examples. The best broad probe used layer 36 and reached validation AUROC 0.8846 and AUPRC 0.8293. This is not enough to claim a general process verifier, but it suggests that hidden states contain useful local algorithmic signals.

Training hidden states, not only probabilities. The current value-function approach trains only an external scalar predictor. A more ambitious direction is to train the student’s representations directly: hidden states near validated correct ideas should become easier to separate from states before the idea, wrong detours, and neutral reasoning. This could be done with a contrastive loss, a lightweight process-credit head, or privileged distillation from a teacher that sees editorials and reference solutions.

The goal is to keep verifier reward as the final correctness signal, but make credit assignment more local, earlier, and less dependent on surface text.

6 Conclusion

The principal conclusion of this project is that the final outcome alone is insufficient for Codeforces RLVR credit assignment. Verifier reward supplies a reliable global label, but it does not distribute credit well across long reasoning and code trajectories.

Solution-conditioned process credit uses privileged information to assess trajectories produced by the student, rather than to generate an ideal answer for it. The student remains an unprivileged solver. A teacher with access to an editorial or correct solution evaluates where important local decisions occur in the student’s already generated work.

The sparse OPD teacher is a useful baseline because it operates on student support and penalizes unnecessary movement from the public distribution. The residual-support parameterization makes this particularly clean: a zero residual exactly recovers the public distribution.

The experiments support the motivation but do not close the problem. Ordinary GRPO uses a meaningful outcome signal while distributing it too coarsely. Privileged teachers learn moderate accepted-versus-rejected separation, and the value teacher produces useful audit heatmaps. None of these observations alone proves faithful causal process credit. The next milestone is to evaluate outcome quality, sparsity, and location quality together against manually reviewed reasoning spans.

References

- [1] Chen et al. The Unlearnability Phenomenon in RLVR for Language Models, 2026. Preprint.
- [2] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process Reinforcement through Implicit Rewards, 2025. [arXiv:2502.01456](#).
- [3] Aristotelis Lazaridis, Dylan Bates, Aman Sharma, Brian King, Vincent Lu, and Jack FitzGerald. EDGE-OPD: Internalizing Privileged Context with Evidence Guided On-Policy Distillation, 2026. [arXiv:2605.23493](#).
- [4] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. In *International Conference on Learning Representations*, 2024. [arXiv:2305.20050](#).
- [5] Mingyang Song and Mao Zheng. A Survey of On-Policy Distillation for Large Language Models, 2026. [arXiv:2604.00626](#).
- [6] Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-Distilled RLVR, 2026. [arXiv:2604.03128](#).
- [7] Qinan Yu, Alexa Tartaglino, Peter Hase, Carlos Guestrin, and Christopher Potts. Outcome Rewards Do Not Guarantee Verifiable or Causally Important Reasoning, 2026. [arXiv:2604.22074](#).
- [8] Huining Yuan, Zelai Xu, Huaijie Wang, Xiangmin Yi, Jiaxuan Gao, Xiao-Ping Zhang, Yu Wang, Chao Yu, and Yi Wu. Verifiable Process Rewards for Agentic Reasoning, 2026. [arXiv:2605.10325](#).
- [9] Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free Process Rewards without Process Labels, 2024. [arXiv:2412.01981](#).

OPD Sparse Teacher

Idea And When It Helps

The useful teacher behavior is local. The teacher should not rewrite the whole trajectory into an expert solution. It should look at the student’s current prefix and use privileged information to slightly reweight the next-token alternatives at moments where the direction matters.

Examples of useful behavior:

- If the student is about to continue with `dynamic programming`, but privileged solution-side information points to a segment tree approach, the teacher should lower the local probability of the `dynamic` branch and raise the probability of a `segment / tree` branch if those tokens are available in the support.
- If the student reasons left-to-right, but the privileged solution invariant is naturally right-to-left, the teacher should move probability near the first branch point where `left / prefix` choices compete with `right / suffix` choices.
- If the student already made the decisive mistake, the teacher should not smear negative reward over the whole remaining code. The valuable signal is near the mistake or near the next recoverable branch.
- If the token is only filler, such as `Wait`, `Hmm`, or generic connective text, the best teacher usually should stay close to the student distribution.

This is why the target is a **sparse privileged residual teacher**: movement should be small at most prefixes, but meaningful at a small number of reasoning-critical prefixes.

In the current residual-support version, the teacher can directly reweight only:

$$K_t \cup \{y_t\}.$$

So this method is most useful when the better local continuation is already among the student’s plausible alternatives. If the correct token is outside the cached support, the current teacher can still lower the sampled bad token, but it cannot directly raise the specific missing alternative.

1. Goal

OPD should train the student on states the student actually visits. For a Codeforces problem statement:

x = public problem statement,

the public student produces a trajectory:

$$y_{1:T} \sim \pi_0(\cdot \mid x).$$

At prefix $y_{<t}$, the teacher evaluates the student’s actual next token y_t . This is important because student trajectories contain detours, partial mistakes, style tokens, and code paths that an expert model would not necessarily generate by itself.

The privileged teacher can condition on:

$$(x, z, y_{<t}),$$

where z is solution-side information. The student/reference side must never see z .

The central token log-ratio is:

$$\rho_t = \log p_t(y_t) - \log q_t(y_t),$$

where:

$$p_t(\cdot) = \pi_{\text{teacher}}(\cdot \mid x, z, y_{<t}), \quad q_t(\cdot) = \pi_0(\cdot \mid x, y_{<t}).$$

A trajectory score is a normalized sum of these token terms:

$$S(x, z, y) = \frac{\beta}{T^\alpha} \sum_{t=1}^T \rho_t.$$

Final verifier labels supervise this sum. The token terms become an implicit process reward, but they are underidentified: many different token-level assignments can explain the same accepted/rejected label.

2. First Baseline: Full-LM Privileged Teacher

The direct baseline trains a full privileged teacher distribution:

$$\pi_\theta(\cdot \mid x, z, y_{<t}).$$

For the sampled student token:

$$\rho_t^{\text{out}} = \log \pi_\theta(y_t \mid x, z, y_{<t}) - \log \pi_0(y_t \mid x, y_{<t}).$$

The outcome loss is a margin loss:

$$\mathcal{L}_{\text{out}} = \ell(m - cS), \quad c = 2l - 1,$$

where $l = 1$ for accepted trajectories and $l = 0$ for rejected trajectories.

To discourage arbitrary distribution drift, the trainer also penalizes sparse movement away from the public student/reference distribution. For each prefix, cache:

$$K_t = \text{TopK}_k(\pi_0(\cdot \mid x, y_{<t})),$$

and use the support:

$$\mathcal{S}_t = K_t \cup \{y_t\}.$$

All vocabulary outside $\mathcal{S}_t$ is represented by one bottom bucket $\perp$. The coarsened movement is a KL over $\mathcal{S}_t \cup \{\perp\}$, and the sparse penalty is:

$$\mathcal{L}_{\text{sparse}} = \lambda \sum_t ((d_t + \epsilon)^p - \epsilon^p), \quad 0 < p < 1.$$

The full objective is:

$$\mathcal{L} = \mathcal{L}_{\text{out}} + \mathcal{L}_{\text{sparse}}.$$

This baseline is scientifically meaningful, but it is hard. The privileged prompt changes the whole model distribution before learning starts:

$$\pi_\theta(\cdot \mid x, z, y_{<t}) \neq \pi_0(\cdot \mid x, y_{<t}).$$

So movement does not naturally start from zero. The model must both use z and imitate the public student distribution. In practice, full-LM runs learned some accepted/rejected ranking signal, especially on train-like validation, but held-out problem validation stayed moderate and heatmaps were noisy.

3. Current Method: Residual-Support Teacher

The residual-support method changes the parameterization. Instead of asking the teacher to produce a full next-token distribution, it starts from the cached student probabilities and learns only a privileged residual correction on the selected support.

At each prefix, the cache contains:

$$\pi_0(v \mid x, y_{<t}), \quad v \in \mathcal{S}_t,$$

where:

$$\mathcal{S}_t = K_t \cup \{y_t\}.$$

The omitted vocabulary mass is preserved as:

$$q_{0,t}(\perp) = 1 - \sum_{u \in \mathcal{S}_t} \pi_0(u \mid x, y_{<t}).$$

The teacher reads the privileged prompt and produces residual logits only for selected tokens:

$$\Delta_{\theta,t}(v), \quad v \in \mathcal{S}_t.$$

The bottom bucket has no learned residual:

$$\Delta_{\theta,t}(\perp) = 0.$$

The residual teacher distribution is:

$$q_{\theta,t}(v) = \frac{\pi_0(v \mid x, y_{<t}) \exp(\Delta_{\theta,t}(v))}{Z_{\theta,t}}, \quad v \in \mathcal{S}_t,$$

and:

$$q_{\theta,t}(\perp) = \frac{q_{0,t}(\perp)}{Z_{\theta,t}}.$$

The normalizer is:

$$Z_{\theta,t} = q_{0,t}(\perp) + \sum_{u \in \mathcal{S}_t} \pi_0(u \mid x, y_{<t}) \exp(\Delta_{\theta,t}(u)).$$

The key property is identity at zero. If all residual logits are zero, then:

$$Z_{\theta,t} = 1,$$

and therefore:

$$q_{\theta,t}(v) = \pi_0(v \mid x, y_{<t}), \quad v \in \mathcal{S}_t,$$

while:

$$q_{\theta,t}(\perp) = q_{0,t}(\perp).$$

So movement starts exactly at the cached student distribution. Training only learns where privileged information should reweight the student's plausible next-token alternatives.

For the sampled student token, the residual log-ratio is:

$$\rho_t = \log q_{\theta,t}(y_t) - \log \pi_0(y_t \mid x, y_{<t}) = \Delta_{\theta,t}(y_t) - \log Z_{\theta,t}.$$

The trajectory score remains:

$$S(x, z, y) = \frac{\beta}{T^\alpha} \sum_{t=1}^T \rho_t.$$

The residual-support movement KL has a compact form:

$$d_t = \sum_{v \in \mathcal{S}_t} q_{\theta,t}(v) \Delta_{\theta,t}(v) - \log Z_{\theta,t}.$$

The sparse loss is again:

$$\mathcal{L}_{\text{sparse}} = \lambda \sum_t ((d_t + \epsilon)^p - \epsilon^p),$$

and:

$$\mathcal{L} = \mathcal{L}_{\text{out}} + \mathcal{L}_{\text{sparse}}.$$

4. Implementation Shape

The pipeline has two stages.

First, cache the public student/reference side. The cache stores the response token ids, sampled-token student logprobs, student top-k token ids, and student top-k logprobs. Mathematically this is the public student/reference distribution:

$$\pi_0(\cdot \mid x, y_{<t}).$$

The student side is computed from the ordinary public prompt. It does not see solution-side privileged information.

Second, train the teacher from cached arrays only. The teacher prompt may include privileged solution-side information. Validation and qualitative audits store token-level arrays: teacher logprobs, student logprobs, logprob differences, movement KL, score/loss fields, metadata, and teacher top-k events.

The residual-support implementation can directly edit only:

$$K_t \cup \{y_t\}.$$

Tokens outside this set are collapsed into the bottom bucket. Therefore, if the student writes `dynamic programming` but the better local continuation is `segment tree`, the current residual-support teacher can directly raise `segment` only if `segment` is in the cached student top-k or is the sampled token. This is an implementation tradeoff, not a fundamental OPD limitation.

5. Data And Validation

The training data is built from student rollouts on Codeforces problems with binary verifier labels.

The problem-balanced training split is built problem-by-problem. For each problem, keep only binary-label trajectories and require both accepted and rejected examples. Then sample the same number from both sides:

$$m_p = \min(n_p^+, n_p^-).$$

This avoids the simple shortcut where easy problems mostly have accepted trajectories and hard problems mostly have rejected trajectories.

Validation is intentionally split into several views:

- trajectory-level validation with balanced random trajectories;
- problem-level validation, closer to generalization;
- train-like validation for fit/capacity checks;
- qualitative audits for heatmaps and teacher top-k inspection.

The main distinction is important: train-like validation can show that the model fits the task, but random-problem validation is the stricter generalization check.

6. Representative Results

Full-LM baseline

The smaller full-LM baseline did learn a real accepted/rejected ranking signal:

- trajectory-level AUC: 0.694 last, 0.712 best;
- problem-level AUC: 0.648;
- problem-level pairwise accuracy: 0.642.

The interpretation is mixed. An AUC clearly above 0.5 means the teacher is not random: it often assigns higher scores to accepted trajectories than to rejected ones. However, the problem-level AUC around 0.65 is still moderate, and the signed threshold score remained poorly calibrated. In other words, the model learned a useful outcome-ranking signal, but this did not yet imply reliable process credit.

Takeaway: learns some ranking signal, but the process credit remains noisy.

Larger shuffled full-LM run

A larger shuffled full-LM run improved some ranking metrics:

- trajectory-level AUC: 0.615 last, 0.703 best;
- problem-level AUC: 0.633 last, 0.667 best;
- problem-level pairwise accuracy: 0.685 last, 0.725 best;
- train-like AUC and pairwise accuracy: 1.0.

The important contrast is between train-like fit and problem-level generalization. The perfect train-like numbers show that the model has enough capacity to separate familiar cases. But the problem-level AUC stays only around 0.63–0.67, and pairwise accuracy stays below 0.73. This suggests that the teacher can fit the training-style distribution, while the harder generalization setting remains only moderately solved.

The held-out validation plots show the same pattern from three angles. The score gap becomes positive, the pairwise AUC stays above the random baseline, and the active movement fraction quickly becomes small. Together, these plots support the interpretation that the teacher learns a sparse ranking signal, but not a fully reliable process-credit signal.

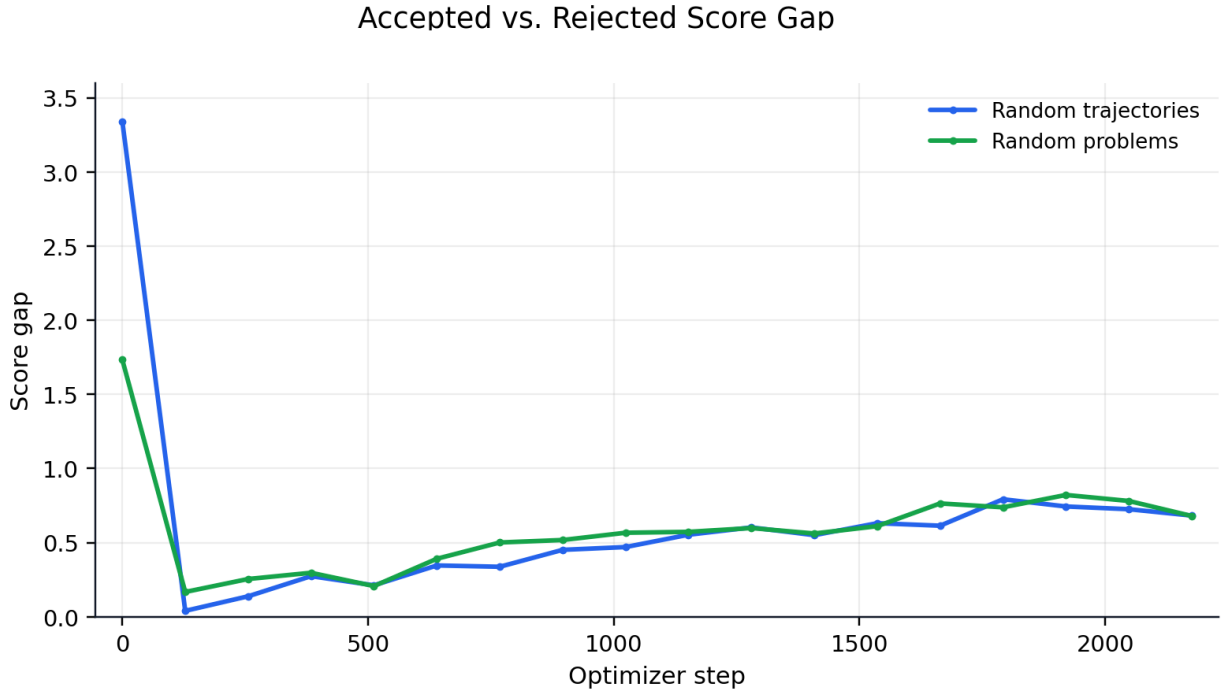


Figure 1: Accepted vs. rejected score gap on held-out validation. After the early transient, both validation views stabilize at a positive gap, meaning accepted trajectories receive higher aggregate teacher scores than rejected trajectories.

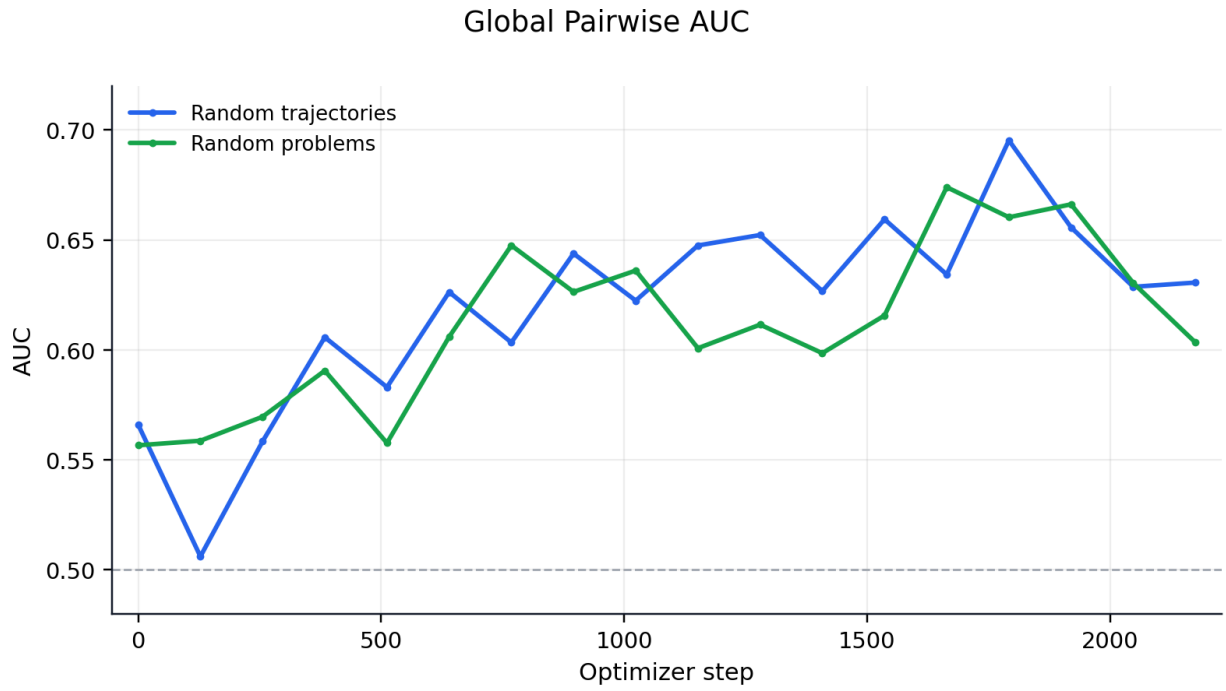


Figure 2: Global pairwise AUC on held-out validation. The curves stay above the random 0.5 baseline and peak around 0.67–0.70, but they remain far from perfect separation.

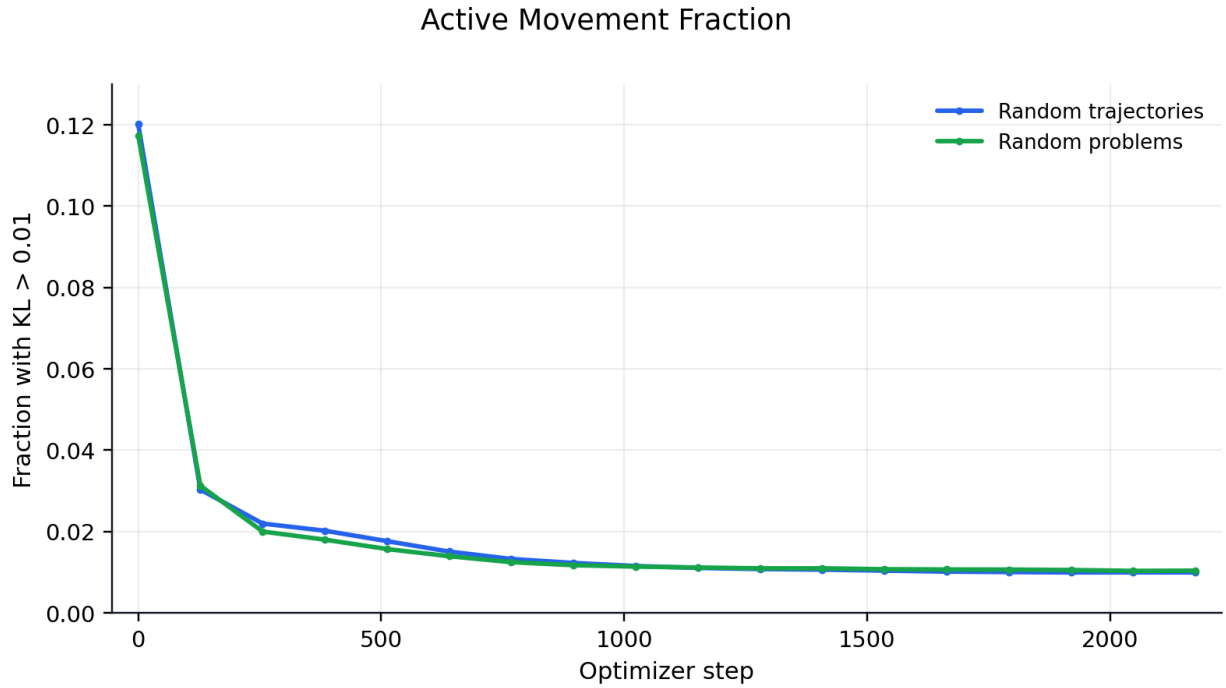


Figure 3: Active movement fraction on held-out validation. The fraction of positions with noticeable KL movement quickly drops toward about one percent, matching the intended sparse-teacher behavior.

Takeaway: strong train-like fitting, but held-out problem generalization remains moderate.

7. Current Conclusion And Problems

The current conclusion is sharper than "token-level credit is noisy". For these reasoning traces, a standalone token often does not have a well-defined semantic label. The object that can be judged is usually a local decision span: a branch, a correction, an implementation commitment, or a persistent false assumption.

This matters because a teacher trained on token movement can otherwise learn the wrong thing: style markers, subword fragments, or late tokens after the real decision has already happened.

Concrete examples from trajectory analyses:

- **1182E: the token is only a fragment of the algorithmic decision.** The relevant reasoning event is: "Compute the exponents and use Fermat's theorem ... how to compute exponents with mod?" In token space this can appear as `Ferm`, and another nearby useful continuation can be `modular`. Neither token is the target by itself. The target decision is: switch to exponent recurrences and compute exponents modulo `MOD-1` because the bases are positive. A reward on `Ferm` alone can teach the teacher to like a word fragment instead of the actual modular-exponent reduction.
- **1560_D: the same not token can support opposite ideas.** In the green local false-start span, `not` is part of the correct repair: "Wait, but that's not possible. Because in order to form `T_str`, we need to have its digits as a subsequence..." Here negation rejects a false worry about the formula. But the same trajectory also uses negation for the wrong temporary claim: "the formula is not correct for all cases," and the later red span keeps treating the correct algorithm as incorrect. Both patterns contain `not`; the difference is what the following reasoning does with the negation. The teacher cannot label `not` when it appears. It must wait to see whether the negation rejects a false concern or rejects the correct method.
- **766_D: the useful correction begins before the obvious phrase.** The trajectory says: "But there's a problem: the find function is defined inside `main()`, which is not allowed in C++ ... Ah, right! C++ doesn't allow nested functions." The important event starts at "But there's a problem", not at `Ah` and not at the token `right`. Rewarding only the later phrase would train on a style marker; the actual credit should cover the whole correction span that moves the code toward a legal standalone `find` function.
- **1814_D: flaw detection is useful only if it changes the plan.** The trajectory says: "this code has a problem ... the loop from 1 to `L_max` is way too slow," but then continues: "perhaps the test cases are designed to pass ... this is the best I can do." The words "has a problem" look good locally, but the full span is bad because the model keeps the too-slow approach. The correct credit target is not the flaw-detection token; it is the decision to either fix the complexity issue or ignore it.

The practical conclusion: token logits are the mechanism we can move, but tokens are not always the right annotation unit. For this project, the teacher should first identify sparse decision spans and only then express that span-level judgment through token-level probability movement. Otherwise the method can optimize visible tokens while missing the reasoning decision that actually determines whether the solution succeeds.